

2. Créer des microservices

Communication entre microservices avec Rabbitmq

Principe de base de RabbitMQ

Dans la partie précédente de ce chapitre, on a mis en œuvre une application microservices basée sur une communication **synchrone** .

Dans cette partie, on va voir comment instaurer un système de communication **asynchrone** entre les services de l'application, en utilisant l'outil **RabbitMQ**.



2. Créer des microservices

Communication entre microservices avec Rabbitmq



Principe de base de RabbitMQ

- **RabbitMQ** est basé sur le protocole « Advanced Message Queuing Protocol » (**AMQP**) ;
- AMQP est un protocole pour les systèmes de messagerie orientés messages (MOM), créé en 2004 ;
- L'objectif d'AMQP est de permettre aux applications **client**, implémentant ce protocole, de communiquer avec un **serveur** de messagerie orienté messages implémentant lui aussi AMQP.
- Le principal avantage du AMQP est qu'il n'impose pas à l'émetteur et au destinataire de comprendre le même langage de programmation.

2. Créer des microservices

Communication entre microservices avec Rabbitmq

Principe de base de RabbitMQ

- **RabbitMQ** est un **message broker** (agents de messages), son rôle est de transporter et de router les messages depuis les **publishers** vers les **consumers**.
- Dans ce modèle, les services client sont représentées par les termes **producteur** et **consommateur**.
- Le rôle du **producteur** est d'envoyer un message au broker à destination du consommateur, on dit qu'il publie (*publish*).
- Le rôle du **consommateur** est de recevoir un message à partir du broker, on dit qu'il consomme (*consume*).
- Le rôle du **broker** est de recevoir les messages envoyés et de les livrer aux destinataires spécifiés.



2. Créer des microservices

Communication entre microservices avec Rabbitmq



Principe de base de RabbitMQ

L'avantage de RabbitMQ réside dans le fait que le **producteur** du message n'a pas à effectuer personnellement l'envoi. Le message **broker** reçoit le message et donne ainsi la possibilité au producteur de commencer une nouvelle tâche. L'émetteur n'est pas obligé d'attendre que le destinataire reçoive le message.

Dans le cadre de cette procédure, le message est placé dans la **file d'attente** avant d'être récupéré par le **consommateur**. À cet instant, l'émetteur est déjà occupé à une nouvelle tâche.

=> Il s'agit donc d'une procédure **asynchrone** : l'émetteur et le destinataire n'ont pas à agir au même rythme.

2. Créer des microservices

Communication entre microservices avec Rabbitmq

Installation du serveur RabbitMQ

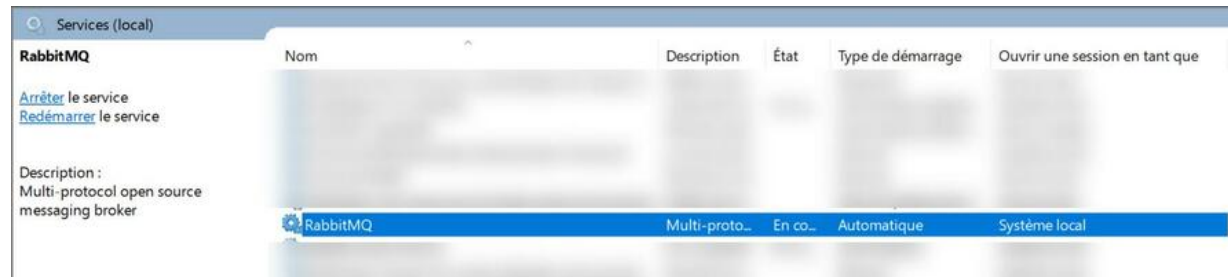
On peut installer RabbitMQ sur n'importe quel système d'exploitation, il suffit de suivre la documentation officielle :

<https://www.rabbitmq.com/download.html>

-> Installation sous Windows:

- Prérequis:

- Avant d'installer RabbitMQ, il faut, tout d'abord, installer **Erlang** (64 bits) en tant qu'Administrateur.
- Erlang peut être téléchargé à partir de ce lien: https://erlang.org/download/otp_versions_tree.html ([Lien direct](#))
- Téléchargez le programme d'installation de RabbitMQ, rabbitmq-server-{version}.exe ([Lien direct](#)) et exécutez-le.
- Ce programme installe RabbitMQ en tant que service Windows et le démarre en utilisant la configuration par défaut.
- On peut vérifier l'installation et le démarrage de RabbitMQ en consultant la liste des services Windows:



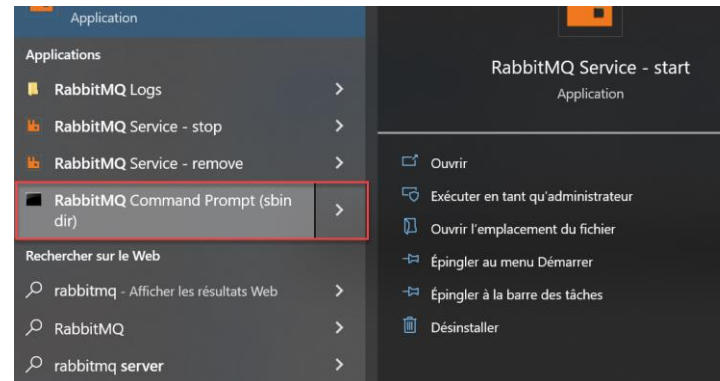
2. Créer des microservices

Communication entre microservices avec Rabbitmq

Installation du serveur RabbitMQ

RabbitMQ offre un plugin de gestion; Ce plugin doit être activé pour qu'il soit fonctionnel ;

- Pour ceci, il faut accéder à l'application « **RabbitMQ Command Prompt** » installée :



- Tapez la commande suivante afin d'activer ledit pulgin :

```
rabbitmq-plugins enable rabbitmq_management
```

- Redémarrez le service

2. Créer des microservices

Communication entre microservices avec Rabbitmq

Installation du serveur RabbitMQ

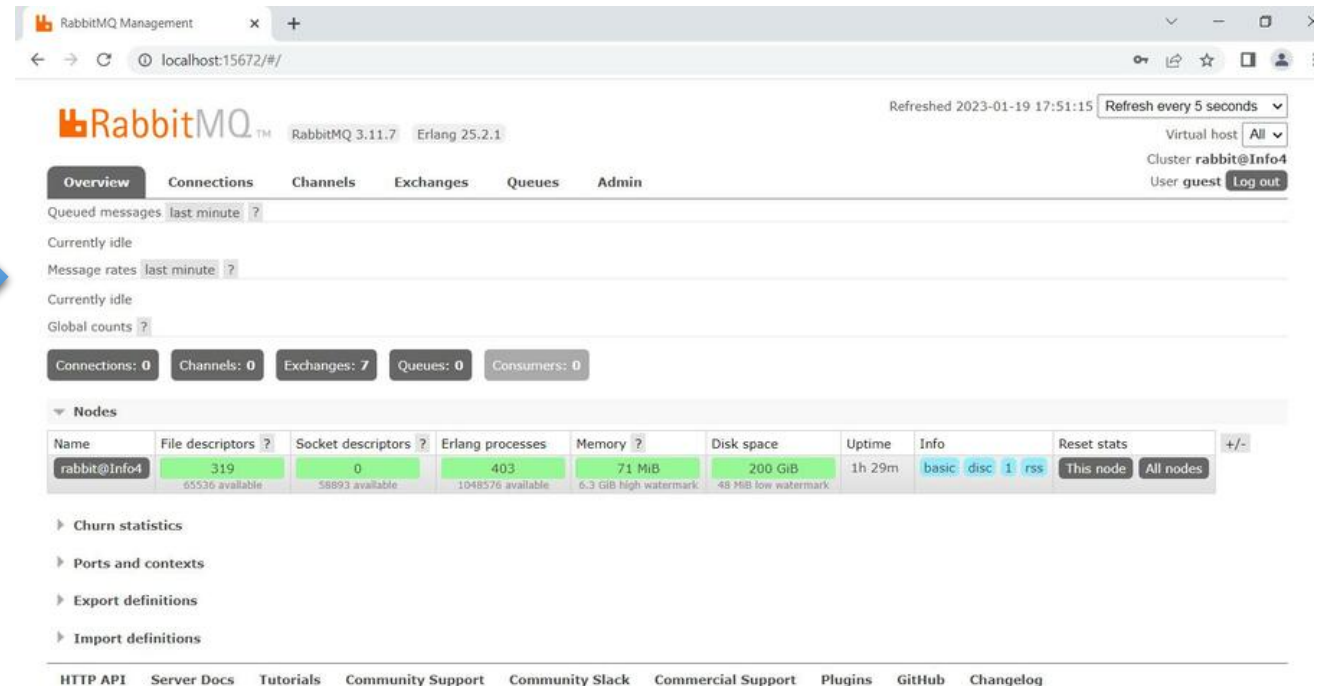
- L'interface utilisateur de gestion est accessible à l'aide d'un navigateur Web à l'adresse `http://{node-hostname}:15672/`

Pour notre cas, on peut utiliser : **`http://localhost:15672`**



Username: *

Password: *



RabbitMQ Management interface showing the Overview tab. The interface displays various metrics and a table of nodes.

Overview | Connections | Channels | Exchanges | Queues | Admin

Queued messages: last minute ?

Currently idle

Message rates: last minute ?

Currently idle

Global counts ?

Connections: 0 | Channels: 0 | Exchanges: 7 | Queues: 0 | Consumers: 0

Nodes

Name	File descriptors ?	Socket descriptors ?	Erlang processes	Memory ?	Disk space	Uptime	Info	Reset stats
rabbit@Info4	319 65536 available	0 58693 available	403 1048576 available	71 MiB 6.3 GiB high watermark	200 GiB 48 MiB low watermark	1h 29m	basic disc 1 rss	This node All nodes

Churn statistics

Ports and contexts

Export definitions

Import definitions

HTTP API | Server Docs | Tutorials | Community Support | Community Slack | Commercial Support | Plugins | GitHub | Changelog

Par défaut :

- Username: guest
- Password: guest

2. Créer des microservices

Communication entre microservices avec Rabbitmq



Installation du module client **amqplib** avec **npm**

- **amqplib** est une bibliothèque Node .js qui implémente les mécanismes nécessaires pour créer des clients AMQP.
- Pour installer cette dépendance, il suffit d'exécuter cette commande: **npm i amqplib**
- On peut vérifier l'installation et la version installé du package sous package.json :

```
  "license": "ISC",  
  "dependencies": {  
    "amqplib": "^0.10.3",  
    "express": "^4.18.2"  
  }  
}
```

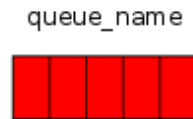

2. Créer des microservices

Communication entre microservices avec Rabbitmq

Mise en œuvre de la communication

Rabbitmq permet d'assurer la communication entre :

- Un **producteur** : le programme qui se connecte au server RabbitMQ et envoie un ou plusieurs messages.
- Une **file d'attente** : le nom de la boîte aux lettres dans RabbitMQ. Bien que les messages transitent par RabbitMQ et vos applications, ils ne peuvent être stockés que dans une file d'attente. Une file d'attente n'est liée qu'aux limites de mémoire et de disque de l'hôte, il s'agit essentiellement d'un grand tampon de messages. De nombreux producteurs peuvent envoyer des messages vers une file d'attente, et de nombreux consommateurs peuvent essayer de recevoir des données d'une file d'attente.
- Un **consommateur** : Le programme qui se connecte au serveur RabbitMQ et consomme les messages.



Notez:

- *le producteur, le consommateur et le serveur ne doivent pas nécessairement résider sur le même hôte.*
- *Une application peut être à la fois producteur et consommateur.*

2. Créer des microservices

Communication entre microservices avec Rabbitmq



Mise en œuvre de la communication

Exemple d'un programme producteur :

Pour que le programme puisse envoyer un message, il faut:

1. Se connecter au serveur ;
2. Créer un canal (Channel : une connexion virtuelle qui permet que chaque opération de protocole effectuée par un client se produit sur un canal particulier. Ainsi, la communication sur un canal est séparée des communications effectuées sur les autres canaux) ;
3. Déclarer une file d'attente ;
4. Envoyer le message au file d'attente.

2. Créer des microservices

Communication entre microservices avec Rabbitmq



Mise en œuvre de la communication

Exemple d'un programme producteur :

```
const amqp = require("amqplib");

var connection, channel;
async function connect() {
  //Se connecter au serveur
  const amqpServer =
    "amqp://localhost:5672";
  connection = await
    amqp.connect(amqpServer);
  //Créer un nouveau canal
  channel = await
    connection.createChannel();

  var queue = 'file_attente1' ;
```

```
  var msg = 'Bonjour le monde' ;
  //Déclarer une file d'attente avec le nom
  "file_attente1"
  await channel.assertQueue(queue);
  //Envoyer le message au file d'attente
  await channel.sendToQueue(queue,
    Buffer.from(msg));
  console.log( " [x] Envoyé %s" , msg);
}
connect();
```

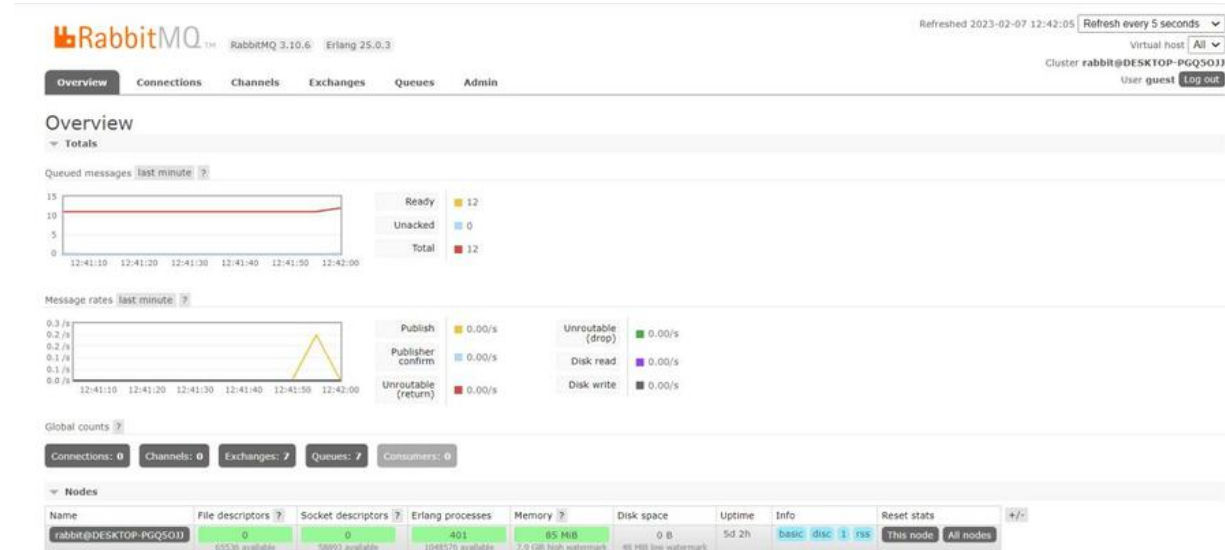
2. Créer des microservices

Communication entre microservices avec Rabbitmq

Mise en œuvre de la communication

Exemple d'un programme producteur :

On peut consulter sur l'interface web de gestion de RabbitMQ un aperçu des différents objets créés:



Finalement, on ferme la connexion et on quitte :

```
setTimeout(function() {  
  connection.close();  
  process.exit(0)  
}, 500);
```

2. Créer des microservices

Communication entre microservices avec Rabbitmq

Mise en œuvre de la communication

Exemple d'un programme consommateur :

```
const amqp = require("amqplib");

var connection, channel;
async function connect() {
  //Se connecter au serveur
  const amqpServer =
    "amqp://localhost:5672";
  connection = await
    amqp.connect(amqpServer);
  //Créer un nouveau canal
  channel = await
    connection.createChannel();

  var queue = 'file_attente1' ;
```

```
    await channel.assertQueue(queue);
    //Recevoir le message depuis la file
    d'attente
    await channel.consume(queue, (msg) => {
      console.log(" [x] Received %s",
        msg.content.toString());
      channel.ack(msg);
    });
    connect();
```

